What is spring

- Spring (Framework) focusses on creating enterprise applications with Java
- Lots of boilerplate code no longer necessary
- Offers IoC (Inversion of Control), transactions
- Spring has very many different parts

Spring vs Springboot

- Spring Boot is built on top of spring framework
- Auto configured
- Custom configuration happens with annotations (@Annotation), XML configuration no longer necessary.
- Easy to set up
- Great for stand alone APIs
- Spring boot simplifies and gives an opinion of how to do it.

Spring Magic

- IoC (Inversion of Control)
- Dependency Injection
- Dependency Inversion
- Spring magic: Spring beans

Spring beans

- Beans have nothing to do with what you think of when you use the word beans in your day to day life.
- A bean is an instance of a class managed by the Spring container.
- A bean is a @Component on a class level.
- A bean is a @Bean on a method level.
- A bean has to be scanned by @SpringbootApplication to be in the scope of the Sprin g singleton.

Spring IOC container

- Part of the core of the Spring Framework
- Responsible for managing all the beans
- Performs dependency injection
- Efficient Dependency Management:
- The IoC container manages object creation and dependency injection, reducing the overhead of manual instantiation and wiring.
- It ensures that objects are created only when needed (lazy initialization) or reused (singleton scope), minimizing resource consumption.
- Lifecycle Management:

    - The container handles the lifecycle of beans, ensuring proper initialization and destruction, which avoids memory leaks and improves application stability.
- Caching and Singleton Optimization:
    - Singleton beans are cached and reused, reducing the cost of repeatedly creating objects.
- Thread-Safe Bean Access:
    - The container ensures thread-safe access to shared resources, improving performance in concurrent environments.
- Configuration Centralization:
    - Centralized configuration reduces redundant code and improves maintainability, indirectly enhancing performance by simplifying debugging and optimization.
- Built-in Features:
    - Features like transaction management, pooling, and scheduling are optimized for performance, reducing the need for custom implementations.

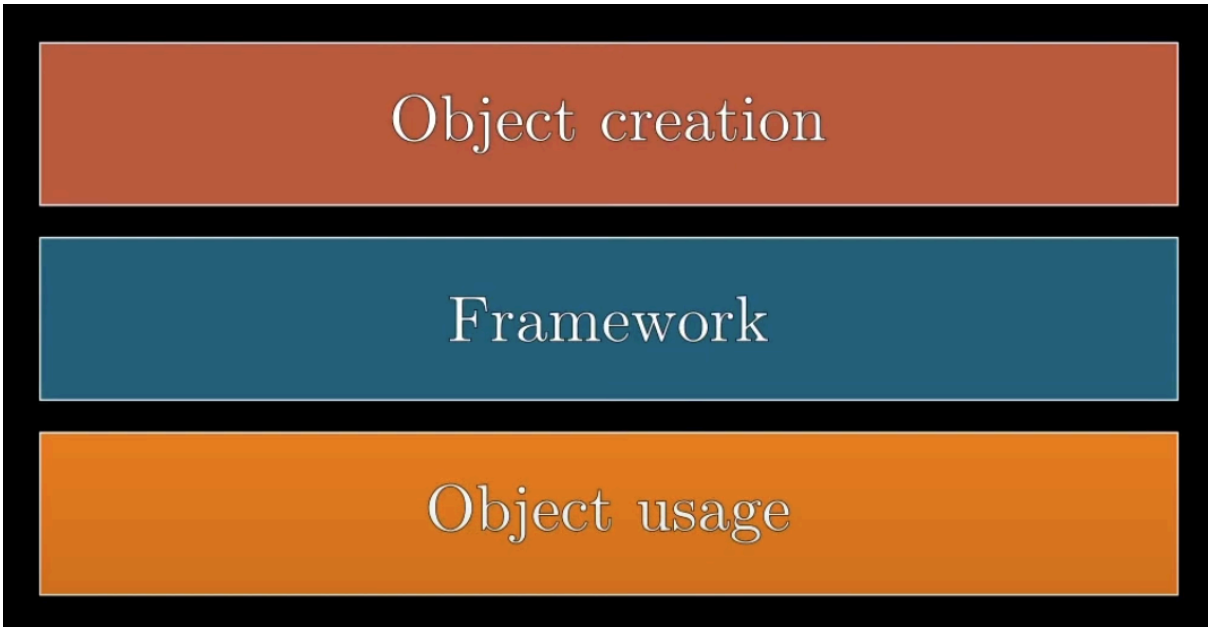
Dependency injection

- It decouples the object creation from its usage.
- It does that by the use of a framework which acts as a layer of abstraction between those actions.

Strong dependency

```java
public class JobQueue {
        private Queue<Job> queue = new PriorityQueue<>();
        private EmailNotifier notifier = new EmailNotifier();

        public void add(Job job) {
                queue.add(job);
                notifier.notify("New job added!");
        }

        public Job remove() {
                notifier.notify("New job removed!");
                return queue.poll();
        }
}
```

```java
public class EmailNotifier {
        private final EmailService service;

        public EmailNotifier() {
                this.service = new EmailService();
        }

        public void notify(String message) {
                this.service.sendEmail(message);
        }
}
```



- The framework contains 2 components:
    - **Graph**: An object that contains a graph of all dependencies on your project
    - **Containers**: Places where dependencies are created by the programmer.
    - **Wirings**: Instructions that tell dependencies injection framework how to manage dependencies.
- *A dependency is referred as an object allocated to another object*

- Before dependency injection, (Engine dependency into Car)

```java
public class Car {
        private final Engine engine;

        public Car(Engine engine) {
                this.engine = engine;
        }
}
```

```java
public class Main {
        public static void main(String[] args) {
                Car car = new Car(new Engine());
        }
}
```

```java
public class Engine {
        public Engine() {

        }
}
```

- After dependency injection (Abstraction between Engine and Car to avoid dependency)

```java
public class Car {
        private final Engine engine;

        @Inject
        public Car(Engine engine) {
                this.engine = engine;
        }
}
```

```java
import com.google.inject.AbstractModule;
import com.google.inject.Provides;

public class CarModule extends AbstractModule {
        @Provides
        public Engine provideEngine() {
                return new Engine();
        }
}
```

```java
public class Main {
        public static void main(String[] args) {

                Injector injector = Guice.createInjector(
                        new CarModule()
                );

                Car car = injector.getInstance(Car.class);

        }
}
```

```java
public class Engine {
        public Engine() {

        }
}
```

Bibliography:

- Spring ultimate basics: What are Spring Beans and what is the Spring Container?
- What is Dependency Injection? (with Java examples)